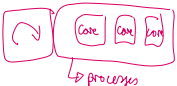
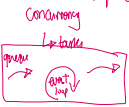
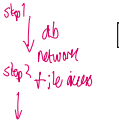
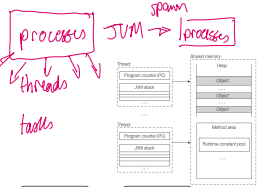
1. Understand the concept of concurrency in java and its importance in software development.
2. Differentiate between process-based concurrency and thread-based concurrency.
3. Identify the states of a thread in java and explain their significance.
4. Implement concurrency in java using the Thread class and the Runnable interface.
5. Describe the four common states of a thread in java (New, Runnable, Blocked/Waiting, Terminated).

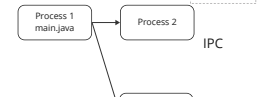












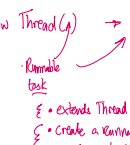


- creating the new threads
- telling them what to do

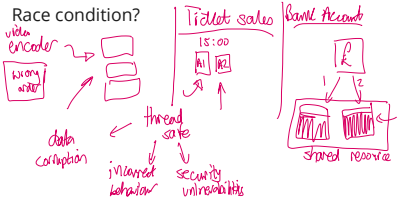
lightweight processes?

task that a thread has do

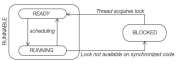
Runnable



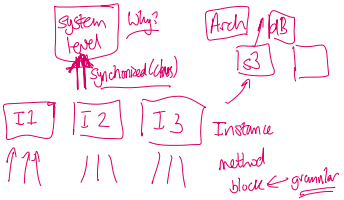
1. Define race condition and recognize its significance in software development, particularly in scenarios involving shared resources and concurrent access.
2. Identify real-life examples illustrating race conditions, such as bank account transactions and online ticket booking, and understand the risks and consequences associated with race conditions in software applications.
3. Develop the ability to write and execute simple Java programs without synchronization to demonstrate race conditions, and analyze the unpredictable behavior resulting from race conditions.
4. Explore synchronization mechanisms in Java, including the synchronized keyword, locks, mutexes, and atomic operations, and comprehend their usage for ensuring thread safety and preventing race conditions.
5. Enhance Java programs with synchronization to mitigate race conditions, utilizing the synchronized keyword and other synchronization mechanisms.







1. Synchronized → method → block
2. Locks → ReentrantLock / Semaphore / ReadWriteLock
3. Atomics → AtomicInteger

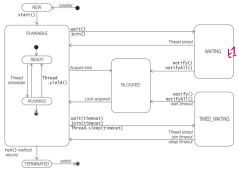


Atomic → primitives!

ReentrantLock

ReentrantLock
ReadWriteLock
Semaphore

ConcurrentHashMap
LinkedQueue
CopyOnWriteArrayList







Outstanding Questions

Important takeways

As you arrive, grab a post-it or two and capture one or two things from yesterday:

- Any outstanding questions?
- Any think you want to make sure you remember going forward?

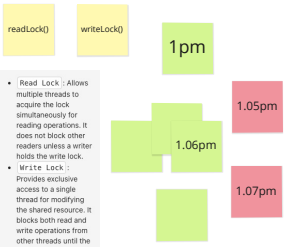
As you write, only you will be able to see your post-it content.

Link to the board is in the chat :)

1. Understand Locks: Learn how locks help manage access to shared resources in programs with multiple threads, ensuring that only one thread can access the resource at a time.
2. Explore ReentrantLock: Discover the benefits of using ReentrantLock in Java, which offers more control and flexibility compared to the traditional synchronized keyword.
3. Use ReentrantLock: Practice using ReentrantLock to protect critical sections of code, where only one thread should be allowed to execute at a time, preventing issues like data corruption.
4. Differentiate ReadWriteLock: Understand the difference between read and write operations on shared resources, and how ReadWriteLock allows multiple threads to read simultaneously while ensuring exclusive access for writing.
5. Learn about Semaphores: Discover semaphores, which are used to control access to a limited number of resources, and how they help prevent resource overload in multithreaded programs.

ReentrantLock

ReadWriteLock



- `Read Lock` : Allows multiple threads to acquire the lock simultaneously for reading operations. It does not block other readers unless a writer holds the write lock.
- `Write Lock` : Provides exclusive access to a single thread for modifying the shared resource. It blocks both read and write operations from other threads until the lock is released.

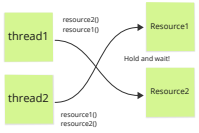
Semaphores

- Permits: max number of threads
- Acquire/Release: acquire() release()
- Counting (fixed number) and binary (0/1)
- Controlling access to resources
- Preventing resource starvation

Compare/Contrast

Feature/Aspect	synchronized Methods/Blocks	Atomic Classes (AtomicInteger, AtomicBoolean, etc.)	ReentrantLock	ReentrantReadWriteLock	Semaphore
Basic Concept	Intrinsic lock for synchronizing code blocks or methods, ensuring only one thread executes at a time.	Lightweight, lock-free, atomic operations on individual variables.	Explicit lock with more features and control than synchronized.	Read-write lock, allowing multiple readers but only one writer at a time.	Limits access to a resource pool with a fixed number of permits.
Concurrency Support	Mutual exclusion, one thread can access a block/method at a time.	Lock-free, concurrent access to individual variables.	Supports mutual exclusion with finer-grained control (e.g., fairness, interruptibility).	Allows multiple threads to read concurrently but write operations are exclusive.	Controls the number of threads accessing a resource concurrently.
Locking Type	Implicit (managed by JVM)	Lock-free (no traditional locking)	Explicit (must acquire and release manually)	Explicit (must acquire and release manually)	Explicit (acquire and release permits manually)
Reentrancy	Yes	Not applicable	Yes (ReentrantLock is reentrant by design)	Yes (ReadLock and WriteLock are reentrant)	Not reentrant by default (need custom logic for reentrancy).
Fairness Policy	No fairness control	Not applicable	Optional fairness (new ReentrantLock(true))	Optional fairness for both read and write locks	No fairness, unless implemented manually.
Interruptibility	No	Not applicable	Supports interruptible lock acquisition (lockInterruptibly())	Write lock supports interruptibility, but reading threads holding a lock cannot be interrupted.	Can block a thread until a permit is available. acquire() is interruptible.
Lock Timeout Support	No	Not applicable	Yes (tryLock(timeout))	Yes (tryLock(timeout) on ReadLock and WriteLock)	Yes (tryAcquire(timeout)) to limit wait time for permits.
Read-Write Separation	No (all threads must wait)	Not applicable	No (mutual exclusion only)	Yes, allows concurrent readers while restricting writers.	No (controls access based on permits, not reading/writing).
Use Case - Best For	Simple mutual exclusion where basic synchronization suffices.	Optimistic concurrency control and atomic operations on individual variables.	Advanced locking where features like fairness, timeouts, or interruptibility are needed.	Scenarios with a heavy read-to-write ratio (e.g., caches, shared configurations).	Controlling access to a limited resource pool (e.g., database connections, thread pools).
Performance Consideration	Optimized by JVM, but can block multiple threads unnecessarily if many reads and few writes.	Very lightweight and high-performance due to non-blocking, atomic operations.	Slightly higher overhead than synchronized, but provides more control.	Efficient in read-heavy environments but can degrade if many write operations occur.	Overhead depends on the number of permits and contention.
Granularity of Control	Low (block or method level)	Fine-grained for single variables	High (fine-grained locking control)	High (separate read and write locks)	Medium (controls the number of concurrent threads).
Starvation Possibility	Yes (especially if many threads are competing for the lock).	No starvation, as operations are lock-free.	No, if fairness is enabled.	Possible, if writers are given preference and too many writes occur.	Can cause starvation if too few permits are available and long-running threads hold them.
Deadlock Risk	Yes, especially in nested or complex locking scenarios.	No (no traditional locking).	Yes, if locks are not acquired/released properly.	Yes, particularly in complex read-write scenarios where multiple locks are involved.	Yes, if permits are not released properly, or if circular wait occurs in complex scenarios.
Best Practices/Points to Be Aware Of	- Simple, easy to use for basic mutual exclusion. - Can block too many threads if not used judiciously.- Use for simple cases where atomicity is needed.	- Best for simple atomic updates on counters, booleans, or references.- Very efficient for high-concurrency, low-complexity situations.- No need to manage locks.	- Use when you need interruptibility, timed locks, or fair access.- Requires manual locking/unlocking.- Can lead to deadlocks if unlock is missed.- Use try-finally blocks to avoid lock leaks.	- Ideal for read-heavy situations (e.g., data that is read frequently but rarely written).- Writers may starve readers if writer preference is enabled.- Not suitable for write-heavy applications.- Be aware of deadlock in complex read-write patterns.- Prefer ReentrantLock for write-heavy scenarios.	- Use for limiting access to shared resources.- Can be used to create pools or rate-limit access.- May cause thread starvation if permits are exhausted for long periods.- Permits must be properly acquired and released to avoid deadlock.

1. Define deadlocks and understand their potential impact on software applications.
2. Discuss and understand various strategies for managing deadlocks, including prevention, avoidance, and detection techniques.
3. Get familiar with deadlock detection algorithms, especially the Banker's algorithm, and its application in real systems.
4. Understand the role of interrupts in managing the lifecycle of threads.
5. Gain a thorough understanding of Java's wait(), notify(), and notifyAll() methods.

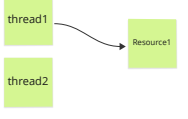


- Careful Lock Ordering
- .tryLock() .tryLock(timeout, unit)
- Bankers Algorithm

the current load

the request

simulation



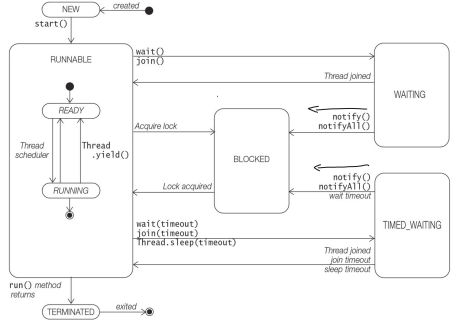
R1 (8) R2 (2) R3 (5)

5

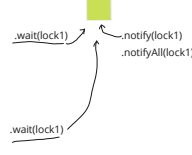
p1

p2

p3



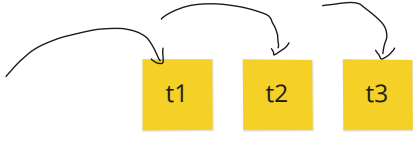
.wait() .notify() -> synchronized()



1. Explore the Java Executor framework and its components for managing thread pools and task execution.
2. Learn about the advantages of using thread pools and scenarios where thread pooling is beneficial in real-world applications.
3. Implement thread pools using Java's Executor framework and explore best practices for configuring and managing thread pools.
4. Identify common pitfalls and performance issues when working with thread pools and learn strategies to avoid them.
5. Gain practical experience through code-along examples demonstrating the implementation of thread pools and utilization of Executor factory methods.

Lab (45)

- Future<> (15-20)
- Mid course review (10-15)



Collections of pre-initialised threads

- Improved performance
- Resource management
- Scalability
- Simplified task management

.start()

.join()

Web servers

Async processing

Batch processing

Executor Interface

.execute(Runnable task)

ExecutorService Interface

- submit
- submit with return values
- control the lifecycle of the executor

ThreadPoolExecutor class

- a pool of worker threads
- Task queuing
- Thread lifecycle management

SingleThread

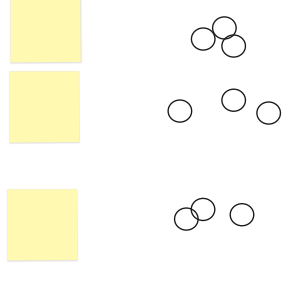
One in, one out
No parallel

CachedThreadPool

Short lived
HTTP
Message processing

FixedThreadPool

Batch processing
CPU-heavy tasks
Thread-limited db connections

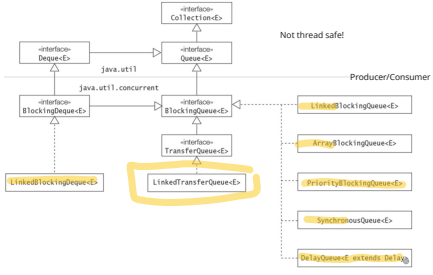


New table

Thread Pool Type	Use Case	Ideal Scenario	Why It's a Good Fit
SingleThreadExecutor Scheduled	Sequential task execution	Logging systems, sequential database transactions, or tasks that need to run one at a time	Guarantees order and prevents concurrency issues
CachedThreadPool	High throughput, short-lived tasks	Web servers handling HTTP requests, message processing queues	Dynamically scales with demand, avoids overhead of thread creation for short tasks
FixedThreadPool Scheduled	Limited, fixed number of concurrent tasks	Batch processing, CPU-bound tasks, limited database connection pools	Controls the number of concurrent threads, prevents resource exhaustion

1. Explain the concept of priority-based task management.
2. Describe how PriorityBlockingQueue implements priority handling in a thread-safe manner.
3. Learn how DelayQueue manages tasks that require execution after a certain delay.
4. Explore how LinkedBlockingDeque allows simultaneous FIFO and LIFO operations.

- ExecutorService + Queues .... don't work too well together!



Not thread safe!

Producer/Consumer

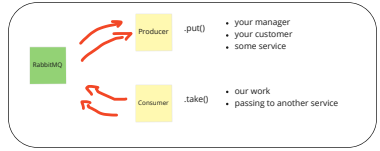
PriorityBlockingQueue

- Task scheduling
- Resource management
- Concurrency control

Task implements Comparable

-> Natural Ordering
-> compareTo(other)

-> Comparator Ordering
-> compare()
-> HIGH, MEDIUM, LOW



- your manager
- your customer
- some service

- our work
- passing to another service

Queues vs Executors

Aspect	Queues (e.g., PriorityBlockingQueue, DelayQueue)	ExecutorService
Task Ordering	Fine-grained control over task ordering (priority, delays, custom ordering)	Tasks executed in submission order (FIFO) unless custom queues are used
Thread Management	Manual thread creation and control, consumer threads block until tasks are available	Automatic thread pooling and reuse, executor manages thread lifecycle
Ease of Use	Requires more complex code to manage threads and tasks	Simple task submission, executor abstracts thread management
Task Scheduling	Useful for priority-based tasks, delayed tasks, and custom consumption (FIFO/LIFO)	Good for periodic or scheduled tasks (via ScheduledExecutorService)
Scalability	Can be scalable but requires custom thread management	Executors provide built-in scalability with thread pools
Real-Life Example	Handling urgent tasks in an order-processing system (priority), handling delayed tasks (delays)	High concurrency applications like web servers, background processing

DelayQueue

<> ScheduledExecutor

Task implements Delayed

-> getDelay()
-> compareTo()

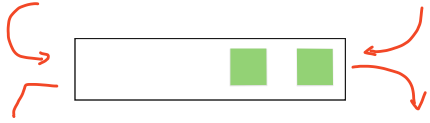
New table

Scenario	Choose ExecutorService	Choose Concurrent Queue (e.g., PriorityBlockingQueue, DelayQueue)
Real-time transaction processing	Ideal for high-frequency or real-time processing, such as transaction analysis or fraud detection. The tasks are executed as they come in, with auto-scaling of threads.	If transaction priority (e.g., risk level) determines which transactions should be processed first, PriorityBlockingQueue offers fine-grained control.
Delayed or scheduled tasks	ScheduledExecutorService is useful for periodic tasks like generating reports, daily batch jobs, or routine alerts.	If you need to delay individual tasks for varying durations (e.g., loan approval after 24 hours), DelayQueue would be a better fit.
Task prioritization	Executor does not provide native priority support. Tasks are handled in FIFO order, suitable for tasks without specific priority.	Best for priority-based tasks. For example, in risk management or resource allocation scenarios, tasks are processed by priority.
Custom task consumption	Executors provide simplicity when you want to handle tasks in parallel without special ordering or delay.	Queues like LinkedBlockingDeque are ideal for situations where some tasks need stack behavior (LIFO) and others need queue behavior (FIFO).

LinkedBlockingDequeue

Dequeue not unqueue

Double Ended Queue



- Work-stealing
- Queue (FIFO)/Stack(LIFO)



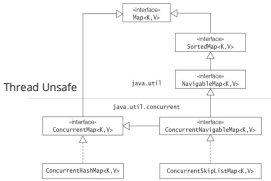
.putFirst() .putLast()
.takeLast() .takeFirst()

.pop() .push()
.shift() .unshift()

Comparing the three queues we've looked at

Feature	PriorityBlockingQueue	DelayQueue	LinkedBlockingDeque
Purpose	Manages elements based on priority in a thread-safe way. elements ordered either by natural order or a provided comparator.	Holds elements that can only be taken after a specified delay, used for scheduling tasks.	Double-ended queue supporting both FIFO and LIFO operations, thread-safe.
Blocking Behavior	Blocks consumer threads when no elements are available.	Blocks consumer threads until delay expires on elements.	Blocks producer/consumer threads when full or empty (optional bounded capacity).
Ordering	Natural ordering (e.g., numerics, strings) or custom comparator-based ordering.	Orders elements based on expiration time (delayed tasks).	No internal ordering, operates on both ends of the deque.
Use Case	- Task scheduling based on priority- Resource allocation where tasks are prioritized.	- Job scheduling- Managing task or resource delays (e.g., token expiration).	- Dual-mode operations (stack/queue) in producer-consumer scenarios.- Work-stealing algorithms.
Thread-Safe	Yes, with no need for external synchronization.	Yes, with no need for external synchronization.	Yes, headtail locks provide safe concurrent access.
Capacity Limit	Unbounded (size limited by available memory).	Unbounded (size limited by available memory).	Optionally bounded (can be set to limit size).
Blocking Methods	put(), take(), waits if queue is empty/full.	put(), take(), waits if delay is not expired or queue is full/empty.	putFirst(), putLast(), takeFirst(), takeLast(), blocking methods.
Common Scenarios	- Task execution based on priority in a multithreaded environment.- Prioritized resource allocation (e.g., CPU jobs).	- Timed execution of tasks.- Resource expiry, token expiry management, task scheduling.	- Work-stealing algorithms- Complex producer-consumer setups with both stack and queue needs.
Key Classes/Interfaces	- PriorityBlockingQueue- Comparator- Comparable	- DelayQueue- Delayed (interface)	- LinkedBlockingDeque- Supports both stack (LIFO) and queue (FIFO) operations.
Example	Processing tasks based on priority in a web server handling requests.	Scheduling tasks to execute after a delay, like session timeout or token expiry.	Managing tasks where operations can be inserted or removed from both ends, like in a job queue.

1. Explain the basic functionality of a HashMap.
2. Discuss why standard HashMap is not safe for use in concurrent scenarios.
3. Describe what data inconsistency means in the context of concurrent computing.
4. Explain the features of ConcurrentHashMap that make it suitable for concurrent use.
5. Compare ConcurrentHashMap with HashMap to highlight how it prevents data inconsistency.



Thread Unsafe



Lock stripping